Full Stack Development Question Bank Solution

Unit 4

(2 marks)

1. **What is Angular ? Why was it introduced?**

Angular is a TypeScript-based open-source front-end web application framework developed and maintained by Google. It's designed to make both the development and testing of such applications easier by providing a framework for client-side MVC (Model-View-Controller) and MVVM (Model-View-ViewModel) architectures, along with the support for two-way data binding.

Angular was introduced to address the challenges and complexities associated with developing single-page applications (SPAs) and dynamic web applications. SPAs are web applications that load a single HTML page and update the content dynamically as the user interacts with the application, without requiring a full page reload. However, as these applications became more complex, managing the structure, organization, and data flow became challenging.

2. **What is Type-script?**

TypeScript is a programming language developed by Microsoft that extends JavaScript by adding static types. It is a superset of JavaScript, which means that any valid JavaScript code is also valid TypeScript code. However, TypeScript introduces additional features and enhancements, primarily focused on improving the development experience and catching errors at compile-time rather than runtime.

3. **What is data binding?What type of data binding angular deploy?**

Data binding is a mechanism that establishes a connection between the application's user interface (UI) and its underlying data model. It allows for the automatic synchronization of data between the two, so changes in one are reflected in the other. In the context of web development, data binding is particularly relevant for dynamic and interactive user interfaces.

Angular supports two types of data binding: one-way data binding and two-way data binding.

4. Difference between angular and AngularJs?

- **AngularJS (1.x):** AngularJS is built on the MVC (Model-View-Controller) architecture. In AngularJS, controllers are responsible for handling user input and updating the model, which in turn updates the view.
- **Angular (2 and later):** Angular uses a component-based architecture. Components are the building blocks of an Angular application and encapsulate both the view and the controller (now called a component class). The architecture is more modular and promotes better organization of code.

5. What are decorators in Angular?

In Angular, decorators are a design pattern that allows you to annotate or modify classes, class properties, and class methods. Decorators provide a way to add metadata to a class or its members, and they are heavily used in Angular to configure and enhance various aspects of the application.

6. Mention some advantages of Angular?

Some key advantages of Angular include:

1. **Modularity:** Angular promotes a modular architecture through the use of components.
2. **Two-Way Data Binding:** Angular provides powerful two-way data binding, allowing automatic synchronization between the model and the view.
3. **Dependency Injection:** Angular's built-in dependency injection system makes it easy to manage and inject dependencies into components and services.
4. **TypeScript:** Angular is built with TypeScript, a statically typed superset of JavaScript.

7. What are templates in angular?

In Angular, templates are an essential part of building the user interface of an application. A template is responsible for defining the structure of views and

how data is presented to users. Angular uses HTML templates enhanced with additional syntax and features specific to the framework.

8. What are annotation's in Angular?

In Angular, annotations are commonly referred to as decorators. Decorators are a design pattern in TypeScript (and in JavaScript with some proposals) that allows you to annotate and modify classes, class properties, and class methods. In the context of Angular, decorators are extensively used to add metadata and configure various aspects of the application.

9. What are directives in angular?

In Angular, directives are a powerful feature that allows you to extend the functionality of HTML elements, attributes, or classes. Directives are used to create reusable components, enhance the behavior of elements, and manipulate the Document Object Model (DOM) within an Angular application. There are three main types of directives in Angular: structural directives, attribute directives, and component directives.

10. What are components in Angular?

In Angular, components are the basic building blocks of a user interface. A component is a self-contained, reusable unit that encapsulates a specific set of functionality, including the associated HTML template and styles, as well as the logic that governs the behavior of that template. Components play a central role in Angular applications, enabling the creation of modular and maintainable code.

11. What are pipes in Angular?

In Angular, pipes are a feature that allows you to transform and format data before it is displayed in the template. Pipes are used to apply various operations on data, such as formatting dates, converting text to uppercase or lowercase, filtering lists, and more. Angular provides a set of built-in pipes, and you can also create custom pipes to meet specific application requirements.

12. What is an ng-module?

In Angular, an NgModule (short for "Angular Module") is a way to organize and structure an Angular application. It is a decorator function that takes a metadata object to describe how the application should be compiled, and it is typically associated with a TypeScript class.

13.What are filters in angular?

Angular filters help with the formatting of the value of an expression that is displayed to the user with no change to the original format. An example is if you want a string in lowercase or uppercase, which can be achieved using the AngularJS filters.

14.What are controllers?

In AngularJS, controllers were an essential part of the architecture, responsible for managing the scope and business logic of a particular section of the application.

15.What is string interpolation in Angular?

String interpolation in Angular is a way to embed expressions within double curly braces (**{{ }}**) in the template, allowing you to dynamically display the values of expressions or variables in the rendered HTML. It provides a convenient and concise syntax for incorporating dynamic data into the view.

16.What is a module in Angular?

In Angular, a module is a mechanism to organize and structure an application into logical units. An Angular module is a TypeScript class annotated with the **@NgModule** decorator. It serves as a container for different parts of an application, such as components, directives, services, and other modules. Modules help in organizing code, managing dependencies, and promoting modularity in an Angular application.

17.What is a service in Angular?

In Angular, a service is a TypeScript class that encapsulates a specific piece of functionality or provides a common set of functions that can be shared across components, directives, or other services. Services are used to organize and share code, separate concerns, and facilitate the reuse of functionality in an Angular application.

18.What is a model in Angular?

In Angular, the term "model" refers to the data structure or object that represents the application's domain and is used to manage and manipulate the application's data. Models are a fundamental part of the Model-View-Controller (MVC) architecture, which Angular follows. The model represents

the state and behavior of the application's data, and it is typically manipulated by services or components.

19.What is the difference between Angular and Reactjs?

- **Angular:** Angular is a full-fledged MVC (Model-View-Controller) framework. It provides a more opinionated and structured architecture out of the box.
- **React:** React is a library for building user interfaces. It focuses on the view layer and leaves the choice of architecture to the developer. Commonly, React applications follow a component-based architecture.

20.Advantages of Angular?

Answer same as 6

(5 marks)

21.What are the advantages and disadvantages of AngularJs?

AngularJS, the first version of the Angular framework, introduced by Google, had its set of advantages and disadvantages. It's important to note that AngularJS is now considered a legacy technology, and its successor, Angular (also known as Angular 2+), has addressed many of its limitations. Nevertheless, let's explore some of the advantages and disadvantages of AngularJS:

Advantages:

1. **Two-Way Data Binding:**
    - AngularJS introduced a powerful two-way data binding feature, where changes in the model automatically reflect in the view, and vice versa. This simplifies the code and reduces the need for manual DOM manipulation.
2. **Dependency Injection:**
    - AngularJS has a built-in dependency injection system, making it easy to manage and inject dependencies into components. This promotes modularity and testability.
3. **Directives:**

- Directives in AngularJS allow the creation of reusable components, enabling developers to extend HTML with custom behaviors. Directives facilitate a declarative approach to building UI components.

Disadvantages:

1. **Performance:**
    - AngularJS can face performance issues, especially with large applications, due to its two-way data binding approach and the need for digest cycles to check for changes in the data model.
2. **Steep Learning Curve:**
    - AngularJS has a steeper learning curve compared to some other front-end frameworks. Understanding concepts like dependency injection, directives, and two-way data binding may take time for new developers.
3. **Not Mobile-Friendly:**
    - AngularJS was primarily designed for desktop web applications. While efforts have been made to improve mobile support, it may not be as effective as some other frameworks in the mobile space.

22.Describe MVC in reference to AngularJs?

In the context of AngularJS, as well as many other web development frameworks, the term MVC refers to the Model-View-Controller architectural pattern. This pattern helps organize the structure of an application by separating it into three main components: the Model, the View, and the Controller.

4. **Model:**
    - The Model represents the data and business logic of the application. It defines how data is stored, processed, and manipulated. In AngularJS, the Model can be thought of as the application's JavaScript objects and data structures, often managed within AngularJS services.

```
angular.module('myApp').service('userService', function() {
  var users = [{ id: 1, name: 'John' }, { id: 2, name: 'Jane' }];
```

```
  return {
    getUsers: function() {
      return users;
    },
    addUser: function(user) {
      users.push(user);
    }
  };
});
```

5. **View:**
    - The View is responsible for presenting the data to the user and handling the user interface. It represents the HTML, CSS, and user interface elements that users interact with. In AngularJS, the View is typically defined using templates, which are HTML files with embedded AngularJS expressions, directives, and bindings.

```
<div ng-controller="UserController">
  <ul>
    <li ng-repeat="user in users">{{ user.name }}</li>
  </ul>
</div>
```

6. **Controller:**
    - The Controller acts as an intermediary between the Model and the View. It handles user inputs, processes them, and updates the Model accordingly. In AngularJS, controllers are JavaScript functions or objects defined using the **controller** directive. They contain the application's business logic and are responsible for managing the state of the Model.

```
angular.module('myApp').controller('UserController',
function($scope, userService) {
  $scope.users = userService.getUsers();

  $scope.addUser = function(name) {
    var newUser = { id: $scope.users.length + 1, name: name };
    userService.addUser(newUser);
  };
});
```

23.What ids are currently used for the development of AngularJs?

The Angular team at Google encourages developers to migrate to newer versions of Angular, specifically Angular (commonly referred to as Angular 2 and above). Angular represents a complete rewrite and is different from AngularJS in terms of architecture, syntax, and features.

For developing with Angular (versions 2 and above), the primary tools, libraries, and concepts used are as follows:

7. **Angular CLI (Command Line Interface):**
    - The Angular CLI is a powerful command-line tool that simplifies the process of creating, building, testing, and deploying Angular applications. It provides a set of commands to generate components, services, modules, and more.
8. **Angular Modules:**
    - Angular applications are organized into modules using the **@NgModule** decorator. Modules help in encapsulating and structuring different parts of an application, making it more modular and maintainable.
9. **TypeScript:**
    - Angular is built with TypeScript, a superset of JavaScript that adds static typing and other features. TypeScript is the preferred language for developing Angular applications.

10. **Components:**

- Components are the building blocks of Angular applications. They encapsulate the logic, data, and view for a specific part of the UI. Components are typically created using the **@Component** decorator.

11. **Directives:**

- Directives in Angular are used to extend the behavior of HTML elements. They can be attribute directives or structural directives. Examples include **ngIf**, **ngFor**, and custom directives created using the **@Directive** decorator.

24.What are directives in AngularJS, explain few of them?

In AngularJS, directives are a powerful feature that allows you to create custom behaviors in your HTML by extending the functionality of existing HTML elements or creating new ones. Directives are markers on a DOM element that tell AngularJS to attach a specific behavior to that element or transform the DOM structure. Here are a few commonly used AngularJS directives:

12. **ngModel:**

- The **ngModel** directive is used for two-way data binding. It binds the value of an HTML element to a property on the associated AngularJS controller. It is commonly used with form elements like input, textarea, and select.

```
<input type="text" ng-model="username">

<p>{{ username }}</p>
```

13. ngRepeat:

- The **ngRepeat** directive is used for repeating a set of HTML elements for each item in a collection. It's often used to generate lists or tables based on an array.

```
<ul>

  <li ng-repeat="item in items">{{ item.name }}</li>

</ul>
```

14. ngIf:

- The **ngIf** directive conditionally renders or removes an element from the DOM based on the truthiness of an expression.

```
<div ng-if="isUserLoggedIn">
  Welcome, {{ username }}!
</div>
```

15. ngClick:

- The **ngClick** directive binds a function to a click event on an HTML element.

```
<button ng-click="handleClick()">Click me</button>
```

16. ngClass:

- The **ngClass** directive allows you to conditionally apply CSS classes to an element based on the evaluation of expressions.

```
<div ng-class="{ 'active': isActive, 'error': hasError }">Dynamic Class</div>
```

25.What is data binding in AngularJs?

Data binding in AngularJS is a powerful feature that establishes a connection between the application's model (the data) and the view (the UI). It allows for automatic synchronization between the model and the view, ensuring that changes in one are reflected in the other without manual intervention. There are two main types of data binding in AngularJS: one-way binding and two-way binding.

1. **One-Way Data Binding:**
    1. In one-way data binding, information flows in a single direction, either from the model to the view or from the view to the model. AngularJS supports the following one-way data binding mechanisms:
        1. **Interpolation ({{ expression }}):**
            - Interpolation is the most common form of one-way binding. It allows you to embed expressions within double curly braces in the HTML, and the expression is evaluated and replaced with the corresponding value.

- **Expression (ng-bind):**
    - The **ng-bind** directive is an alternative to interpolation. It binds the content of an HTML element to an expression, and the element's content is updated whenever the expression changes.
- **Directives (e.g., ng-src, ng-href):**
    - Some directives in AngularJS, such as **ng-src** and **ng-href**, allow you to bind dynamic values to attributes of HTML elements.

```
<img ng-src="{{ imageUrl }}" alt="Dynamic Image">

<a ng-href="{{ linkUrl }}">Dynamic Link</a>
```

2. Two-Way Data Binding:

- Two-way data binding establishes a bidirectional connection between the model and the view. Changes in the model automatically update the view, and vice versa. The most common example of two-way data binding in AngularJS is using the **ng-model** directive.

```
<input type="text" ng-model="username">

<p>{{ username }}</p>
```

In this example, changes in the input field are immediately reflected in the paragraph below, and vice versa.

Two-way data binding is a powerful feature, but it's important to use it judiciously to avoid unnecessary complexity and performance issues, especially in large applications.

26.What are the controllers?And what I the use of controllers in AngularJs?

In AngularJS, controllers are JavaScript functions or objects that play a crucial role in organizing and managing the application's business logic, data, and user interactions. Controllers are responsible for mediating between the model (data) and the view (UI) in the Model-View-Controller (MVC) architecture. They encapsulate the logic related to a specific part of the application and are responsible for handling user input, updating the model, and interacting with services.

Here are key aspects of controllers in AngularJS:

3. **Definition:**

- Controllers are defined using the **controller** function or the **controllerAs** syntax in AngularJS. The **controller** function takes a name and a function that defines the behavior of the controller.

```
angular.module('myApp').controller('MyController', function($scope) {

});
```

4. Scope:

- The **$scope** object is a special object in AngularJS that serves as the bridge between the controller and the view. It holds the model data and methods that are accessible from the associated view.

```
angular.module('myApp').controller('MyController', function($scope) {
  $scope.message = 'Hello, AngularJS!';
});
```

In the example above, the **message** property on the **$scope** object can be accessed in the associated view.

5. Dependency Injection:

- AngularJS controllers can use dependency injection to access AngularJS services, such as **$http** for making HTTP requests or custom services for business logic.

```
angular.module('myApp').controller('MyController', function($scope, $http) {

});
```

6. Updating the Model:

- Controllers are responsible for updating the model based on user input or other events. They define functions that can be called from the view to modify the data.

```
angular.module('myApp').controller('MyController', function($scope) {
  $scope.counter = 0;


  $scope.incrementCounter = function() {
    $scope.counter++;
```

```
  };
});
```

In the example above, the **incrementCounter** function can be called from the view to increment the **counter** property.

7. Event Handling:

- Controllers can handle events triggered by user interactions, directives, or services. They respond to events by executing specific logic.

```
angular.module('myApp').controller('MyController', function($scope) {
  $scope.handleClick = function() {
    console.log('Button clicked!');
  };
});
```

In this example, the **handleClick** function can be invoked when a button is clicked in the associated view.

27.What is a service?

In the context of web development and frameworks like Angular, a service is a reusable, self-contained piece of code that performs a specific set of functions or provides a particular feature. Services in Angular are used to organize and share code across different parts of an application, promoting modularity, maintainability, and reusability.

Here are key characteristics and purposes of services in the Angular framework:

8. **Modularity:**
    - Services help in breaking down an application into smaller, modular pieces. Each service encapsulates a specific functionality or a set of related functionalities, making the codebase more organized.
9. **Reusability:**

- Services can be reused across different components, directives, or other services. This promotes a DRY (Don't Repeat Yourself) coding philosophy, as common functionality is encapsulated in a service and can be used wherever needed.

10. **Separation of Concerns:**

- Services contribute to the separation of concerns within an application. Business logic, data manipulation, and other functionalities are encapsulated within services, while components focus on the presentation and user interaction.

11. **Dependency Injection:**

- Angular has a built-in dependency injection (DI) system that allows services to be injected into components, directives, and other services. This makes it easy to manage and share dependencies across different parts of an application.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class MyService {
  getData() {
  }
}
```

12. **Singleton Pattern:**

- By default, Angular services are singleton instances. This means that there is only one instance of a service throughout the application, and it is shared among all the components and services that inject it. This ensures a consistent state and behavior.

28.What is routing in AngularJs?

In AngularJS, routing refers to the process of navigating between different views or pages in a single-page application (SPA). The AngularJS framework

provides a built-in module called **ngRoute** that facilitates the implementation of routing. Routing enables the creation of dynamic, client-side navigation without the need to reload the entire page, providing a smoother and more responsive user experience.

Here are the key components and concepts related to routing in AngularJS:

13. **ngRoute Module:**

- To enable routing in an AngularJS application, you need to include the **ngRoute** module. This module provides the necessary directives and services for defining routes and handling navigation.

<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular-route.js"></script>

Ensure that the **angular-route.js** script is included in your project.

14. **Route Configuration:**

- Routes are configured using the **$routeProvider** service, which is part of the **ngRoute** module. You define routes by specifying a URL path, a template (HTML file or inline template), and a controller.

```
angular.module('myApp', ['ngRoute'])
 .config(function($routeProvider) {
  $routeProvider
   .when('/home', {
    templateUrl: 'views/home.html',
    controller: 'HomeController'
   })
   .when('/about', {
    templateUrl: 'views/about.html',
    controller: 'AboutController'
   })
   .otherwise({
    redirectTo: '/home'
```

```
    });
});
```

In this example, two routes (**/home** and **/about**) are defined with their corresponding templates and controllers. The **otherwise** method specifies the default route to navigate to if the requested route is not matched.

15. **View Templates:**

- Views or templates are the HTML files that correspond to each route. These templates define the structure of the page that will be loaded when the route is activated.

```
<div>
  <h1>Welcome to the Home Page!</h1>
</div>


<div>
  <h1>About Us</h1>
  <p>This is the about page.</p>
</div>
```

16. **Controllers:**

- Controllers are associated with each route to provide the logic and data for the corresponding view. The controller is defined in the route configuration.

```
angular.module('myApp')
  .controller('HomeController', function($scope) {
    $scope.message = 'Welcome to the Home Page!';
  })
  .controller('AboutController', function($scope) {
    $scope.message = 'This is the about page.';
  });
```

17. ng-view Directive:

- The **ng-view** directive is a placeholder in the HTML where the views associated with different routes will be injected. It marks the area where the content of the current route will be dynamically loaded.

<div ng-view>

The content of this **ng-view** element will be replaced with the content of the active route.

(10 marks)

29. A Part:-What are the software requirements to create an AngularJs project?

B part:-And what are the steps to create an AngularJs project?

C Part:-And write a program to display hellow world?

A. Software Requirements for AngularJS Project:

Before creating an AngularJS project, you'll need the following software tools:

1. **Text Editor or Integrated Development Environment (IDE):**
    - Choose a text editor or IDE of your preference for writing and editing your code. Examples include Visual Studio Code, Sublime Text, Atom, or WebStorm.
2. **Node.js and npm (Node Package Manager):**
    - AngularJS projects often involve Node.js and npm for package management. Download and install Node.js from https://nodejs.org/, which includes npm.

B. Steps to Create an AngularJS Project:

Follow these steps to create a basic AngularJS project:

1. **Initialize a New Project:**
    - Open a terminal and navigate to the directory where you want to create your project. Run the following command to initialize a new Node.js project:

npm init

Follow the prompts to set up your **package.json** file.

2. Install AngularJS:

- Install AngularJS using npm. Run the following command in the

terminal:

Npm install angular

3. Create Project Files:

- Create an HTML file (e.g., **index.html**) and include AngularJS:

```
<!DOCTYPE html>
<html lang="en" ng-app="myApp">
<head>
  <meta charset="UTF-8">
  <title>AngularJS Hello World</title>
  <script src="node_modules/angular/angular.js"></script>
</head>
<body>


</body>
</html>
```

4. Define AngularJS Module and Controller:

- In the same HTML file, define an AngularJS module and a controller:

```
<body ng-controller="HelloController">
  <h1>{{ greeting }}</h1>

  <script>
    var app = angular.module('myApp', []);
    app.controller('HelloController', function($scope) {
      $scope.greeting = 'Hello, World!';
    });
  </script>
</body>
```

5. Run the Project:

- Open the HTML file in a web browser. You should see the "Hello, World!" message displayed on the page.

C. Program to Display Hello World:

Here's the complete program to display "Hello, World!" using AngularJS:

```
<!DOCTYPE html>
<html lang="en" ng-app="myApp">
<head>
  <meta charset="UTF-8">
  <title>AngularJS Hello World</title>
  <script src="node_modules/angular/angular.js"></script>
</head>
<body ng-controller="HelloController">
  <h1>{{ greeting }}</h1>
  <script>
    var app = angular.module('myApp', []);
    app.controller('HelloController', function($scope) {
      $scope.greeting = 'Hello, World!';
    });
  </script>
</body>
</html>
```

30. A Part: - What is data binding in AngularJS?

B Part: - What is the difference between one way data binding and two-way data binding?

Data binding in AngularJS is a powerful feature that establishes a connection between the application's model (the data) and the view (the UI). It allows for automatic synchronization between the model and the view, ensuring that changes in one are reflected in the other without manual intervention. Data binding simplifies the process of keeping the user interface and the underlying data in sync, providing a more dynamic and responsive user experience.

There are two main types of data binding in AngularJS:

1. **One-Way Data Binding:**
    - In one-way data binding, information flows in a single direction, either from the model to the view or from the view to the model. Changes in the model update the view, but changes in the view do not affect the model directly.

Example of one-way binding (interpolation):

```
<p>{{ message }}</p>
```

In this example, the content of the paragraph is bound to the value of the **message** property in the AngularJS model.

2. Two-Way Data Binding:

- Two-way data binding establishes a bidirectional connection between the model and the view. Changes in the model automatically update the view, and vice versa. This is achieved using the **ng-model** directive in AngularJS.

Example of two-way binding:

```
<input type="text" ng-model="username">

<p>{{ username }}</p>
```

1. In this example, the value of the input field (**username**) is bound to the **username** property in the model. Changes in the input field update the model, and changes in the model update the input field.

B. Difference Between One-Way and Two-Way Data Binding:

1. **Direction of Flow:**
    - **One-Way Data Binding:**
        - Information flows in a single direction, either from the model to the view or from the view to the model.
    - **Two-Way Data Binding:**
        - Information flows in both directions. Changes in the model update the view, and changes in the view update the model.
2. **Syntax:**
    - **One-Way Data Binding:**
        - Typically achieved using expressions and directives like **{{ expression }}** or **ng-bind**.
    - **Two-Way Data Binding:**
        - Achieved using the **ng-model** directive, which binds the value of an input field to a property in the model.
3. **Use Cases:**
    - **One-Way Data Binding:**
        - Suitable for scenarios where you want to display data in the view but don't need to capture user input or update the model based on user interactions.
    - **Two-Way Data Binding:**
        - Ideal for forms and user input scenarios where you want to maintain a real-time connection between the user interface and the underlying data model.
4. **Implementation:**
    - **One-Way Data Binding:**
        - Implemented using expressions or directives in the HTML template.
    - **Two-Way Data Binding:**

- Implemented using the **ng-model** directive, which not only displays the value but also updates the model when the value changes.

Understanding the distinction between one-way and two-way data binding is crucial for designing AngularJS applications effectively, as it influences how data is managed and displayed in the user interface.

31. A Part: - What is service in AngularJS? B Part: - What are the different services in AngularJS?

In AngularJS, a service is a reusable, singleton object or function that provides a specific set of functionalities or performs a certain task. Services play a vital role in organizing and separating concerns in an AngularJS application. They encapsulate common business logic, data operations, or utility functions, making them available for reuse throughout the application.

AngularJS services are frequently used for tasks such as making HTTP requests, sharing data between components, handling application-specific logic, and managing the state of the application. Services promote modularity, reusability, and maintainability by allowing developers to encapsulate and share functionality across different parts of the application.

A. Different Services in AngularJS:

AngularJS provides several built-in services that cover various aspects of application development. Here are some commonly used services in AngularJS:

1. **$http:**
    - The **$http** service is used for making HTTP requests to fetch or send data to a server. It supports various HTTP methods and provides a convenient way to interact with web APIs.

```
angular.module('myApp').controller('MyController', function($http) {
  $http.get('/api/data').then(function(response) {
  });
});
```

2. $log:

- The **$log** service is a simple logging service that provides methods for writing log messages. It is often used for debugging and logging information during development.

```
angular.module('myApp').controller('MyController', function($log) {
  $log.debug('Debug message');
  $log.info('Information message');
  $log.warn('Warning message');
  $log.error('Error message');
});
```

3. $location:

- The **$location** service provides methods for interacting with the browser's URL. It can be used to read or change the URL, navigate to different routes, and handle route parameters.

```
angular.module('myApp').controller('MyController', function($location) {
  $location.path('/new-route');
});
```

4. $routeParams:

- The **$routeParams** service is used to access parameters from the URL when using AngularJS routing. It provides an object containing the values of route parameters.

```
angular.module('myApp').controller('MyController', function($routeParams) {
  var id = $routeParams.id;
});
```

5. $timeout and $interval:

- The **$timeout** service is used to execute a function after a specified delay, while the **$interval** service repeatedly executes a function at specified intervals.

```
angular.module('myApp').controller('MyController', function($timeout,
$interval) {
```

```
$timeout(function() {
  }, 2000);
  $interval(function() {
  }, 1000);
});
```

6. Custom Services:

- Developers can create their own custom services to encapsulate application-specific functionality. Custom services can be injected into controllers, directives, or other services.

```
angular.module('myApp').service('myCustomService', function() {
  this.getData = function() {
  };
});
angular.module('myApp').controller('MyController',
function(myCustomService) {
  var data = myCustomService.getData();
});
```

32.A Part: - What is routing in AngularJS? Part B: -write a program to create home and about page?

Routing in AngularJS refers to the mechanism of navigating between different views or pages in a single-page application (SPA). Instead of loading entire HTML pages from the server, AngularJS allows developers to define routes within the client-side application. Each route is associated with a specific view template and a controller. When users navigate to a different route, AngularJS dynamically updates the content of the page without triggering a full page reload.

AngularJS uses the **ngRoute** module to implement routing. The key components of routing in AngularJS include:

1. **Route Configuration:**

- Developers define routes by configuring the **$routeProvider** service. Routes specify a URL path, a template (HTML file or inline template), and a controller.

```
angular.module('myApp', ['ngRoute'])
  .config(function($routeProvider) {
    $routeProvider
      .when('/home', {
        templateUrl: 'views/home.html',
        controller: 'HomeController'
      })
      .when('/about', {
        templateUrl: 'views/about.html',
        controller: 'AboutController'
      })
      .otherwise({
        redirectTo: '/home'
      });
  });
```

2. View Templates:

- Views or templates are HTML files that correspond to each route. These templates define the structure of the page that will be loaded when the route is activated.

```
<div>
  <h1>Welcome to the Home Page!</h1>
</div>


<div>
```

```
  <h1>About Us</h1>
  <p>This is the about page.</p>
</div>
```

3. Controllers:

- Controllers are associated with each route to provide the logic and data for the corresponding view. The controller is defined in the route configuration.

```
angular.module('myApp')
  .controller('HomeController', function($scope) {
    $scope.message = 'Welcome to the Home Page!';
  })
  .controller('AboutController', function($scope) {
    $scope.message = 'This is the about page.';
  });
```

4. ng-view Directive:

- The **ng-view** directive is a placeholder in the HTML where the views associated with different routes will be injected. It marks the area where the content of the current route will be dynamically loaded.

```
<div ng-view></div>
```

5. Navigation:

- Navigation between routes is typically achieved using links with the **ng-href** directive or the **ng-click** directive to trigger route changes.

```
<a ng-href="#/home">Home</a>
<a ng-href="#/about">About</a>
```

Alternatively, the **$location** service can be used to programmatically change the route.

```
angular.module('myApp')
  .controller('NavController', function($location) {
```

```
  $scope.goToHome = function() {
    $location.path('/home');
  };

  $scope.goToAbout = function() {
    $location.path('/about');
  };
});
```

B. Program to Create Home and About Page:

```
<!DOCTYPE html>
<html lang="en" ng-app="myApp">
<head>
  <meta charset="UTF-8">
  <title>AngularJS Routing Example</title>
  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.js"></script
>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular-
route.js"></script>
</head>
<body>

  <div ng-controller="NavController">
    <a ng-href="#/home">Home</a>
    <a ng-href="#/about">About</a>
  </div>
```

```
<div ng-view></div>

<script>
  var app = angular.module('myApp', ['ngRoute']);

  app.config(function($routeProvider) {
    $routeProvider
      .when('/home', {
        templateUrl: 'views/home.html',
        controller: 'HomeController'
      })
      .when('/about', {
        templateUrl: 'views/about.html',
        controller: 'AboutController'
      })
      .otherwise({
        redirectTo: '/home'
      });
  });

  app.controller('HomeController', function($scope) {
    $scope.message = 'Welcome to the Home Page!';
  });

  app.controller('AboutController', function($scope) {
    $scope.message = 'This is the about page.';
```

```
  });

  app.controller('NavController', function($scope, $location) {
    $scope.goToHome = function() {
      $location.path('/home');
    };

    $scope.goToAbout = function() {
      $location.path('/about');
    };
  });
</script>

</body>
</html>
```

33.What are the expressions in AngularJS?

In AngularJS, expressions are snippets of code that are evaluated in the context of the AngularJS application. They are used to bind data to HTML elements, manipulate data, and control the flow of the application. AngularJS expressions are written using a simple and concise syntax enclosed within double curly braces **{{ }}**.

Here are some key characteristics and use cases of AngularJS expressions:

18. **Data Binding:**

- Expressions are commonly used for data binding, allowing you to dynamically display data in the HTML based on the values in your AngularJS model.

```
<p>{{ message }}</p>
```

In this example, the content of the paragraph is dynamically bound to the value of the **message** property in the AngularJS model.

19. **Arithmetic Operations:**

- Expressions support basic arithmetic operations, allowing you to perform calculations within the HTML template.

```
<p>{{ 2 + 2 }}</p>
```

The result of the expression **2 + 2** will be dynamically displayed in the paragraph.

20. String Concatenation:

- Expressions can concatenate strings, making it easy to display combined text.

```
<p>{{ 'Hello, ' + name }}</p>
```

If **name** is a variable in the AngularJS model, the expression will concatenate it with the string 'Hello, '.

21. Logical Operations:

- Expressions support logical operations, such as comparisons and boolean logic.

```
<p>{{ age > 18 ? 'Adult' : 'Minor' }}</p>
```

In this example, the expression checks if the **age** variable is greater than 18 and displays 'Adult' or 'Minor' accordingly.

22. Function Calls:

- Expressions can call functions defined in the AngularJS controller.

```
<p>{{ getFullName() }}</p>
```

If **getFullName** is a function in the controller, its result will be displayed in the paragraph.

34.What is dependency injection in AngularJS? Explain How it works?

Dependency Injection (DI) is a design pattern in software development that deals with how components or services obtain their dependencies. In the context of AngularJS (and its successor, Angular), dependency injection is a core concept that plays a crucial role in building modular, maintainable, and testable applications.

In AngularJS, dependency injection involves providing components (such as controllers, services, or directives) with the objects or services they depend on. Instead of components creating their dependencies directly, AngularJS's injector system supplies them as part of the component's construction. This promotes loose coupling between components, making them more modular and easier to manage.

Here's an overview of how dependency injection works in AngularJS:

23. **Injector:**

- AngularJS has a built-in injector that is responsible for creating instances of components and managing their dependencies. The injector is a key part of the AngularJS framework that performs dependency injection.

24. **Services:**

- In AngularJS, services are often used as singletons that encapsulate specific functionality or hold shared data. These services can be injected into controllers, directives, and other services that depend on them.

```
angular.module('myApp').service('myService', function() {
  this.getData = function() {
    return 'Data from the service';
  };
});
```

25. Injecting Dependencies:

- Components declare their dependencies by specifying them as parameters in their constructor function. The AngularJS injector takes care of providing the necessary dependencies when creating an instance of the component.

```
angular.module('myApp').controller('MyController', function($scope, myService) {
  $scope.data = myService.getData();
});
```

In this example, **$scope** and **myService** are dependencies injected into the controller.

Dependency injection in AngularJS provides several benefits, including:

- **Modularity:** Components are more modular, and their dependencies can be easily swapped or upgraded.
- **Testability:** Components can be tested in isolation by providing mock or test implementations of their dependencies.

- **Maintainability:** The loose coupling between components makes the codebase more maintainable, as changes to one component do not necessarily impact others.

35. What is the difference between $scope and this in AngularJS controllers? Provide examples where each is used.

In AngularJS, both $scope and this are used for binding data between the controller and the view, but they serve different purposes.

1. $scope in AngularJS:

- $scope is an object used to bind the controller's data to the view. It allows for two-way data binding, meaning updates to the model automatically reflect in the view and vice versa.
- It is automatically available inside the controller.

Example:

```
angular.module('myApp', [])
  .controller('myCtrl', function($scope) {
     $scope.message = "Hello from $scope!";
  });
```

In the HTML view:

```
<div ng-controller="myCtrl">
   <p>{{message}}</p>
</div>
```

In this example, the message is bound to the view using $scope.

2. this in AngularJS:

- this is used to bind properties and methods to the controller itself. It is especially used with the controllerAs syntax for better readability and modularity.
- Unlike $scope, this does not bind directly to the view unless specified with controllerAs.

Example:

```
angular.module('myApp', [])
  .controller('myCtrl', function() {
     this.message = "Hello from this!";
  });
```

In the HTML view:

```
<div ng-controller="myCtrl as ctrl">
   <p>{{ctrl.message}}</p>
```

Here, the message is bound to this using the ctrl alias.

Key Differences:

Aspect	$scope	this
Binding	Two-way data binding	Requires controllerAs syntax
Usage	In controller functions	Used with controllerAs
Scope	Inside the controller's scope	Available on controller instance
Modern Usage	Less common in modern AngularJS	Preferred for modularity and readability

---

36. What is the difference between AngularJS and ReactJS?

Feature	AngularJS	ReactJS
Type	Full-fledged framework	Library focused on UI components
Architecture	MVC (Model-View-Controller)	Component-based architecture
Data Binding	Two-way data binding	One-way data binding (with state management)
Rendering	Uses directives for DOM manipulation	Virtual DOM for efficient rendering
Learning Curve	Steep due to complex features like dependency injection	Relatively easier with a focus on UI components
Dependency Injection	Built-in dependency injection	No built-in DI (third-party libraries required)
Use Case	Suitable for large-scale applications	Ideal for building UIs and SPAs

Key Differences:

1. **AngularJS** is a **full MVC framework** with extensive built-in features, including two-way data binding and dependency injection, making it great for larger applications.
2. **ReactJS**, on the other hand, is primarily a **UI library** focused on

creating dynamic interfaces with a virtual DOM for performance. React is often used with additional libraries for routing, state management, etc.